TCP: Transport Layer

Classic TCP Congestion Control

The algorithm has three major components: (1) slow start, (2) congestion avoidance, and (3) fast recovery. Slow start and congestion avoidance are mandatory components of TCP.

The approach taken by TCP is to have each sender limit the rate at which it sends traffic into its connection as a function of perceived network congestion. If a TCP sender perceives that there is little congestion on the path between itself and the destination, then the TCP sender increases its send rate; if the sender perceives that there is congestion along the path, then the sender reduces its send rate. But this approach raises three questions. **First,** how does a TCP sender limit the rate at which it sends traffic into its connection? **Second,** how does a TCP sender perceive that there is congestion on the path between itself and the destination? And third, what algorithm should the sender use to change its send rate as a function of perceived end-to-end congestion?

We saw that each side of a TCP connection consists of a receive buffer, a send buffer, and several variables (LastByteRead, rwnd, and so on). The TCP congestion-control mechanism operating at the sender keeps track of an additional variable, the congestion window. The congestion window, denoted **cwnd**, imposes a constraint on the rate at which a TCP sender can send traffic into the network. Specifically, the amount of unacknowledged data at a sender may not exceed the minimum of cwnd and rwnd, that is:

LastByteSent – LastByteAcked <= min{cwnd, rwnd}

Q1/ Explain the logic for using the function: min{cwnd, rwnd} ?

Since TCP does not use negative acknowledgments, the receiver cannot send an explicit negative acknowledgment back to the sender about a lost packet.

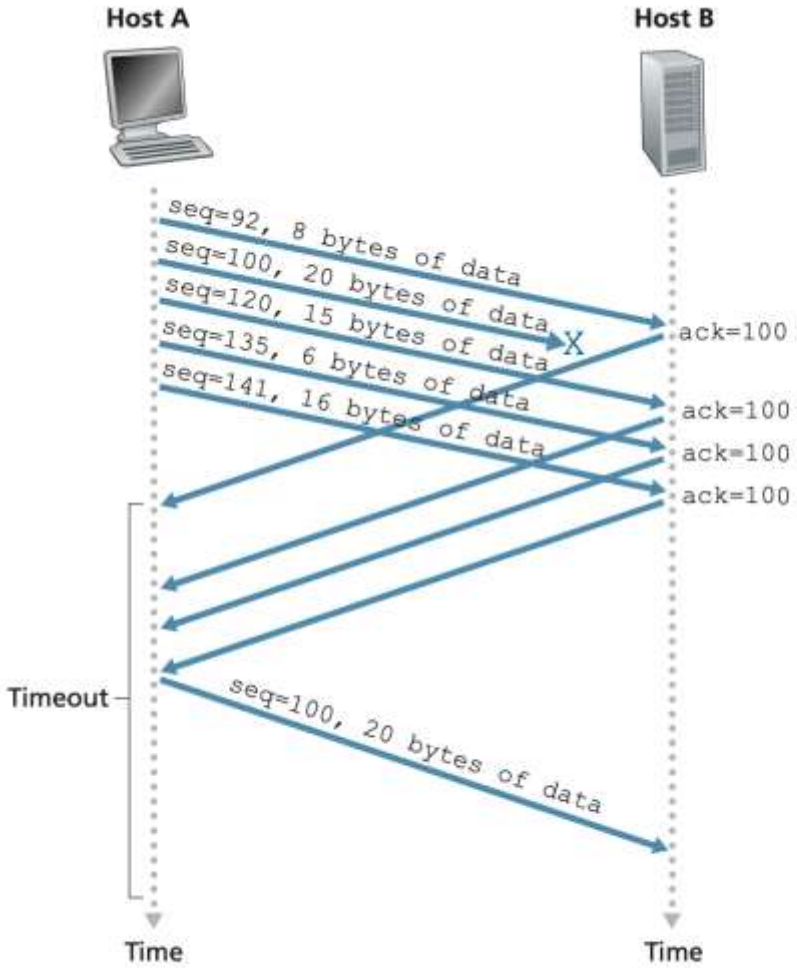
Q2. How does TCP function without negative acknowledgments?

Instead, it simply reacknowledges (that is, generates a duplicate ACK for) the last in-order byte of data it has received. Because a sender often sends a large number of segments back to back (remember rwnd, cwnd), if one segment is lost, there will likely be many back-to-back duplicate ACKs. If the TCP sender receives three duplicate ACKs for the

same data, it takes this as an indication that the segment following the segment that has been ACKed three times has been lost.

Q3. Why do you think it waits for 3 duplicate ACKs rather than 2 duplicate ACKS before declaring a packet as lost?

In the case that three duplicate ACKs are received, the TCP sender performs a fast retransmit [RFC 5681], retransmitting the missing segment before that segment's timer expires. As shown in the figure below:



37 ♦ Fast retransmit: retransmitting the missing segment before the segment's timer expires

Q4. Look at the figure and define what a fast retransmit means?

When there is excessive congestion, then one (or more) router buffers along the path overflows, causing a datagram (containing a TCP segment) to be dropped. The dropped datagram, in turn, results in a loss event at the sender—either a timeout or the receipt of three duplicate ACKs—which is taken by the sender to be an indication of congestion on the sender-to-receiver path.

Having considered how congestion is detected, let's next consider the more optimistic case when the network is congestion-free, that is, when a loss event doesn't occur. In this case, acknowledgments for previously unacknowledged segments will be received at the TCP sender. TCP will take the arrival of these acknowledgments as an indication that all is well—that segments being transmitted into the network are being successfully delivered to the destination—and will use acknowledgments to increase its congestion window size (which essentially means transmission rate from sender will increase). Note that if acknowledgments arrive at a relatively slow rate , then the congestion window will be increased at a relatively slow rate. On the other hand, if acknowledgments arrive at a high rate, then the congestion window will be increased more. So the rate of acknowledgements drives the sender to increase or decrease its sending rate and this is called TCP's **self_clocking.**

The two signs TCP is looking out for:

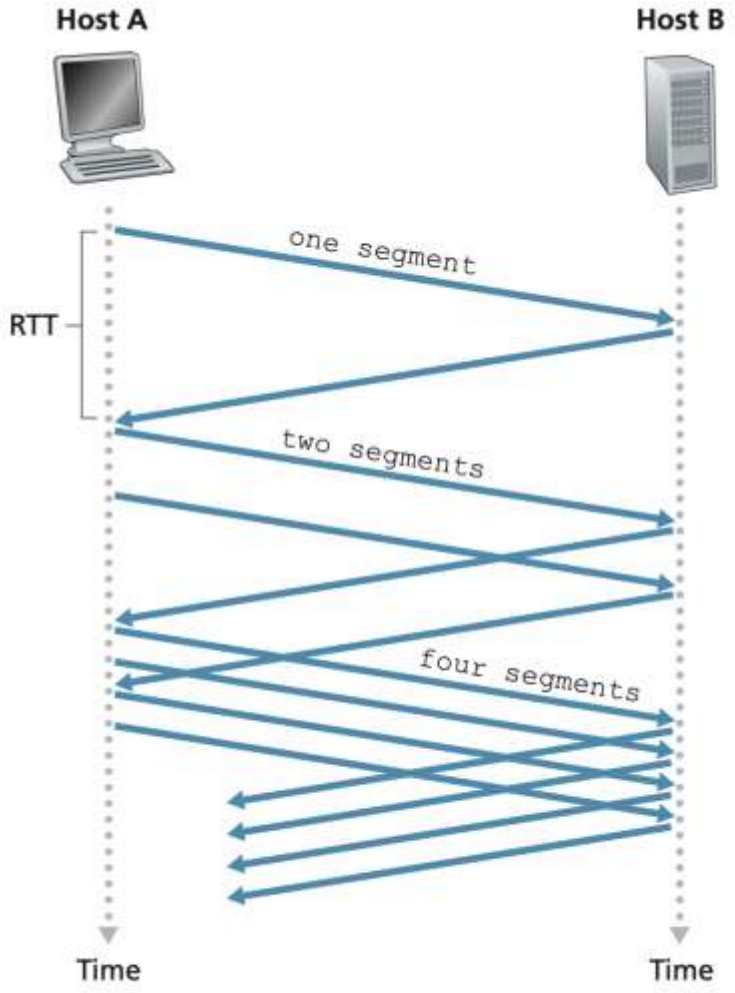
1. Either a segment is lost which implies congestion, and hence, the TCP sender's rate should be decreased when a segment is lost.
2. Or an acknowledgement is received which indicates that the network is delivering the sender's segments to the receiver, and hence, the sender's rate can be increased when an ACK arrives for a previously unacknowledged segment.

Given ACKs indicating a congestion-free source-to-destination path and loss events indicating a congested path, TCP's strategy for adjusting its transmission rate is to increase its rate in response to arriving ACKs until a loss event occurs, at which point, the transmission rate is decreased.

Q5 a) What is self clocking?

b) In which cases does TCP do self clocking?

TCP Slow Start:



- TCP slow start

TCP to begin with sends a packet and waits for an acknowledgment. When this acknowledgment arrives, the TCP sender increases the congestion window by one MSS and sends out two maximum- sized segments. These segments are then acknowledged, with the sender increasing the congestion window by 1 MSS for each of the acknowledged segments, giving a congestion window of 4 MSS, and so on. This process results in a doubling of the sending rate every RTT (round trip time). Thus, the TCP send rate starts slow but grows exponentially during the slow start phase.

But when should this exponential growth end?

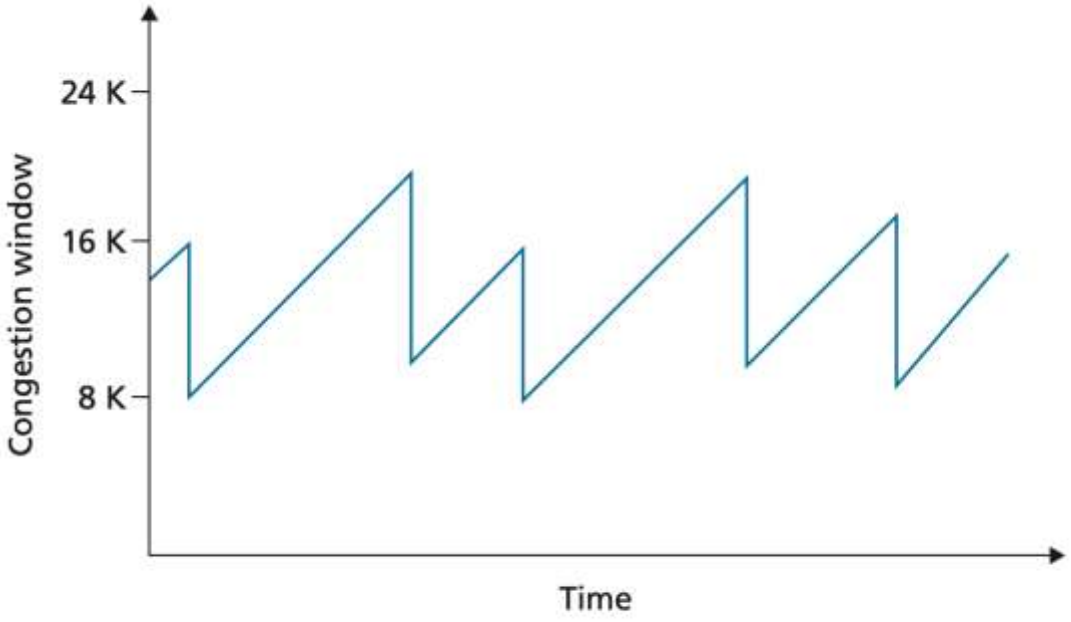
1. **First, if there is a loss event (i.e., congestion) indicated by a timeout for an acknowledgement to a packet. TCP sets *ssthresh* variable to cwnd/2 and cwnd=1. Slow start is restarted.**

2. **When cwnd reaches value in ssthresh slow start phase will end and congestion avoidance will begin.**
3. **When a triple ACK is received, slow start ends; TCP performs a fast retransmit and enters the fast recovery state.**

Q6. What do you think is the variable ssthresh for? Why is it given the value cwnd/2?

TCP congestion-avoidance

On entry to the congestion-avoidance state, the value of ***cwnd*** is approximately half its value when congestion was last encountered—congestion could be just around the corner! Thus, rather than doubling the value of cwnd every RTT, TCP adopts a more conservative approach and increases the value of ***cwnd*** linearly.



3.53 ♦ Additive-increase, multiplicative-decrease congestion control

TCP congestion control is often referred to as an additive-increase, multiplicative-decrease (AIMD) form of congestion control. AIMD congestion control gives rise to the "saw tooth" behavior shown in Figure 3.53

Q7. From all the reading above explain how TCP follows AIMD?